# SWARM RESCUE CHALLENGE 2025
# CIEDS-DRDO

## 31st March 2025

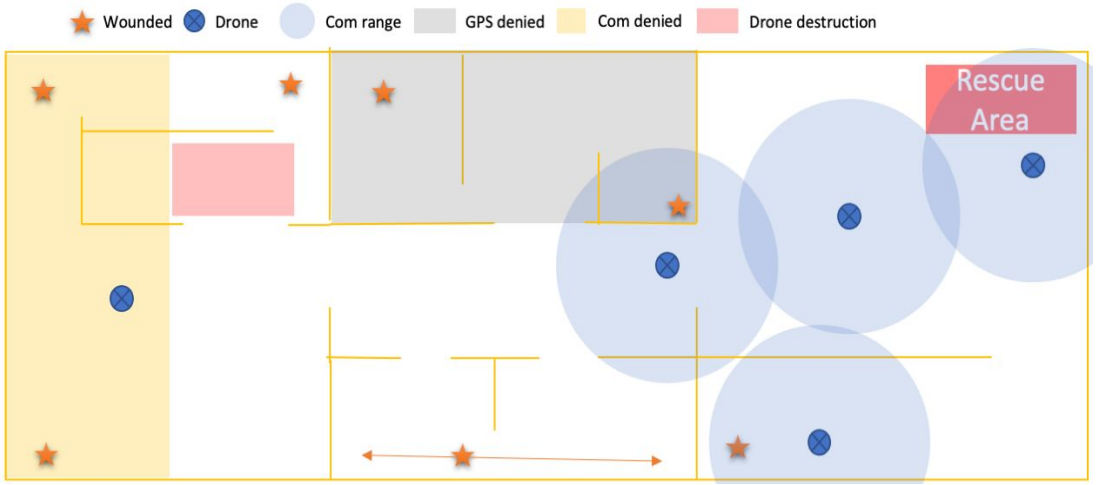
Team Illusion ( Team 07)
IIT Hyderabad

భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

# TEAM ILLUSION

- Neeraj Balachandar
- Kevin DSouza
- Shriram Hari
- A. Padmaprabhan

# THE CHALLENGE




- Exploration of unknown environment through autonomous swarm of drones to locate and rescue wounded people to a rescue center

- The drone swarm is controlled with a limited communication range, a LiDAR, a Semantic sensor, GPS sensor and Compass onboard each drone, all with a finite drone life

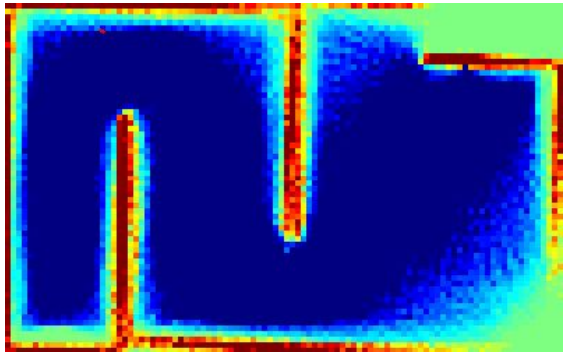
# OBJECTIVES

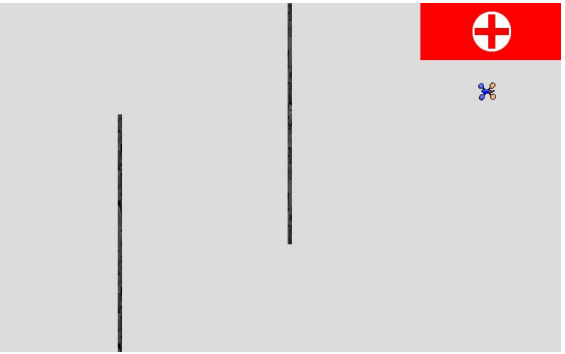
- Maximum exploration of unknown map
- Fusion of Sensors
- Path planning
- Communication and Coordination
- Navigation through special zones
- Maximising life of drones, collision avoidance
- Efficient algorithms

# OUR CONTROLLER

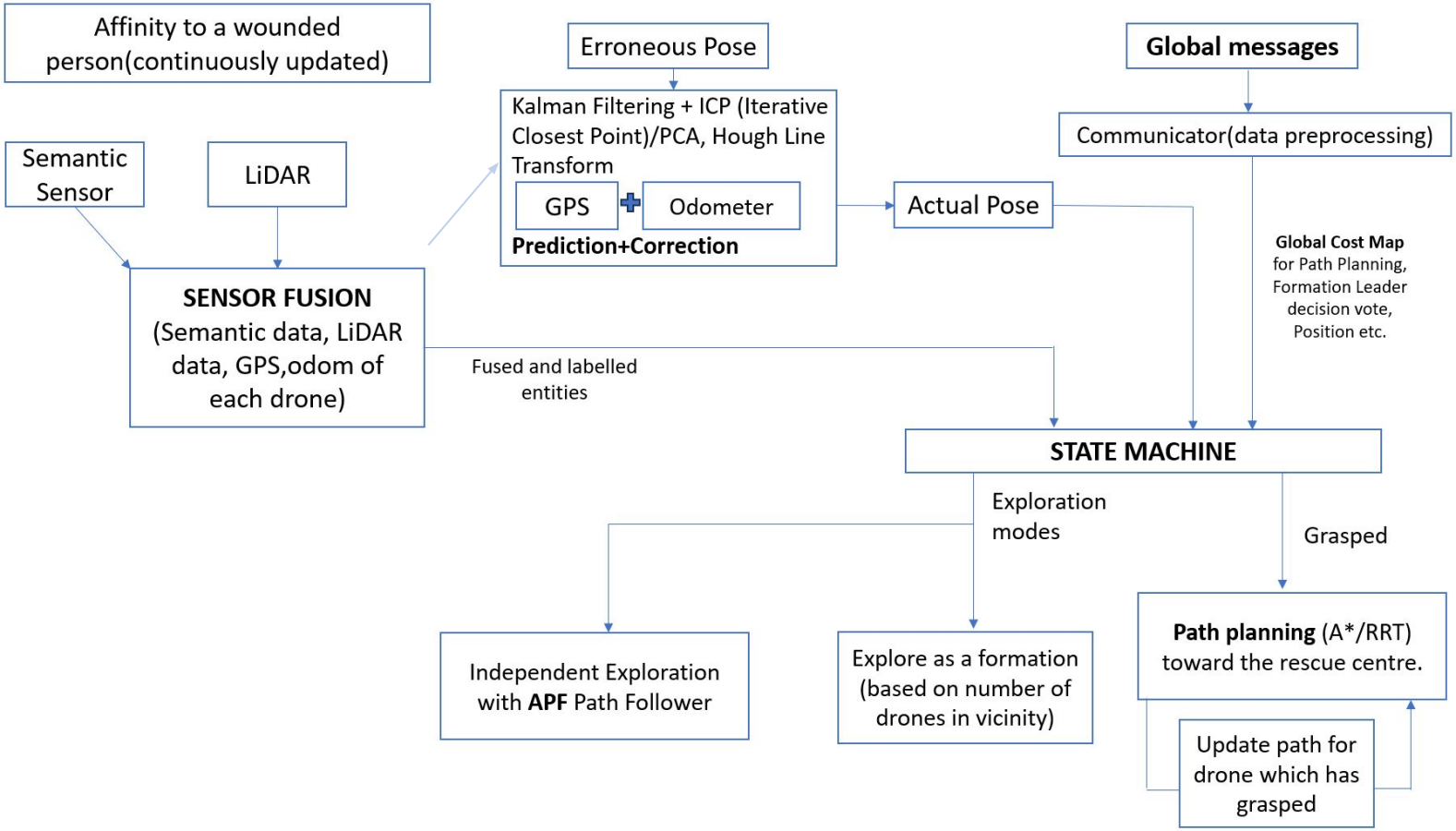
- Our drone controller combines two algorithms, one for the robust exploration of the unknown map and the other as a global path planner.

- The algorithms combined with sensor fusion and the capability to communicate have enabled the drone swarm to achieve exploration and rescue operations at the same time intelligently

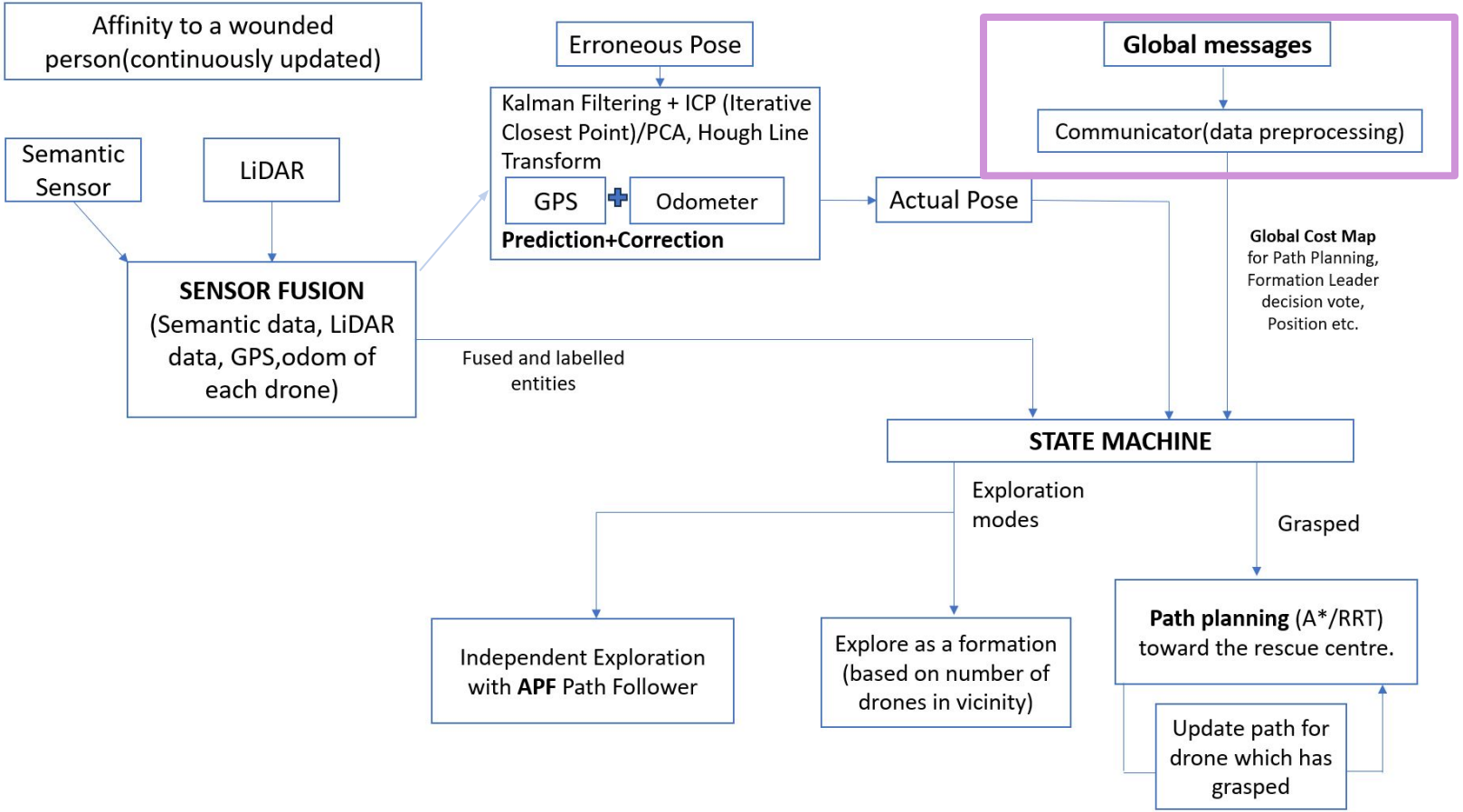




# OUR CONTROLLER - A HIGH LEVEL ARCHITECTURE



# COMMUNICATION



# COMMUNICATION

- Every drone **broadcasts and receives** status periodically
- Each drone sends and receives structured messages containing
  - **Unique ID** of each drone
  - **Score**-based **occupancy map** for obstacle tracking
  - Information on **objects grasped** by the drone
  - **Cost function** related to movement
  - The drone's current **GPS** location under real time processing
- **Condition based updation** of Global occupancy map from each drone
- Updation of GPS values and **synchronization** of **locations** of each drone for **formation** calls
- Communication by **voting** used for resolving grasping conflicts

# STATE MACHINE

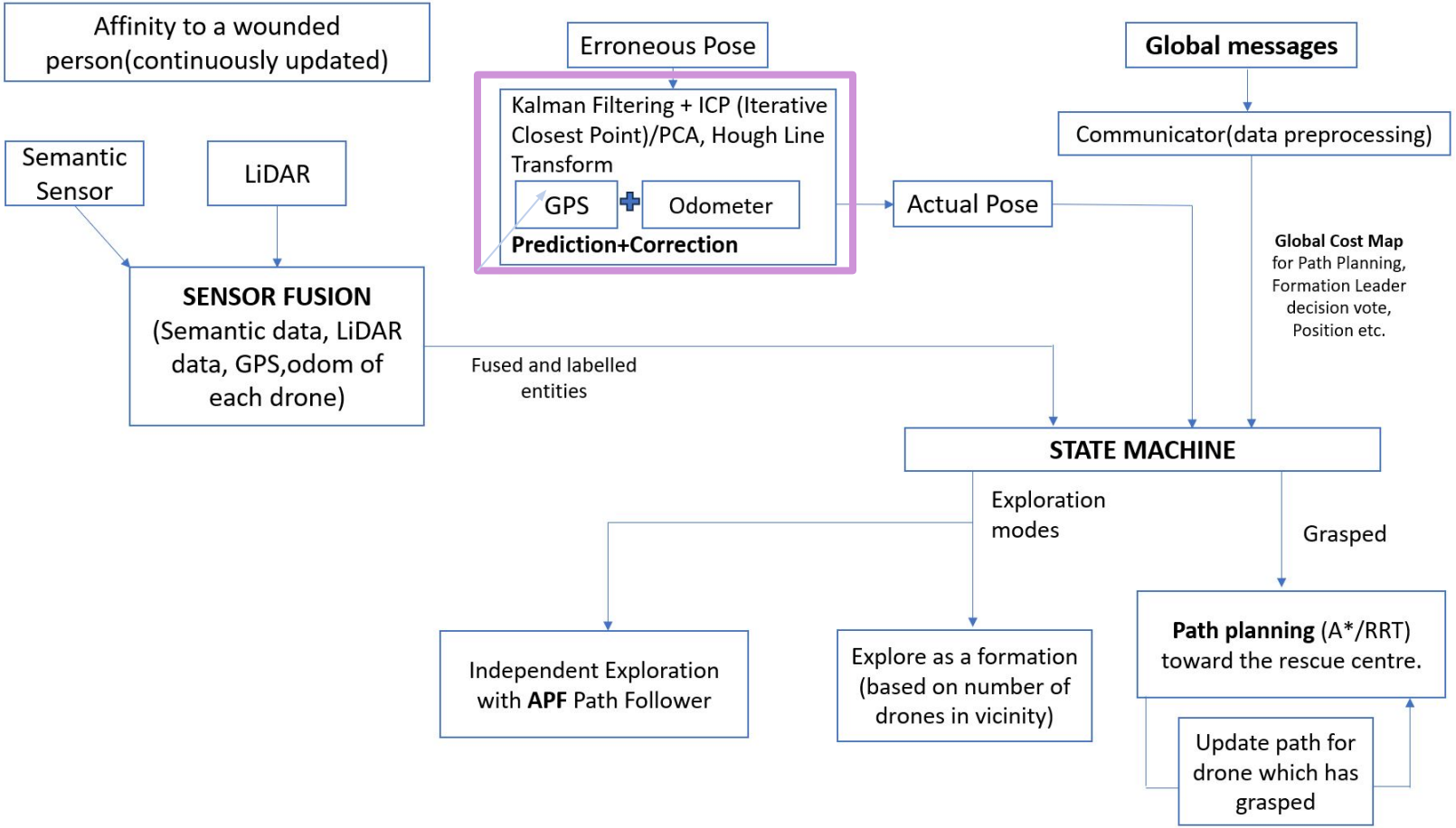
It updates the drone's state based on the current state and sensor inputs.

- **activity**: Current activity/state of the drone
- **found_wounded**: Boolean indicating if a wounded person is found
- **found_rescue_center**: Boolean indicating if a rescue center is found
- **grasped_entities**: Boolean indicating if the drone is holding a wounded entity
- **voteResult**: Boolean for consensus decision-making

**The drones can be in any of the below four states:**

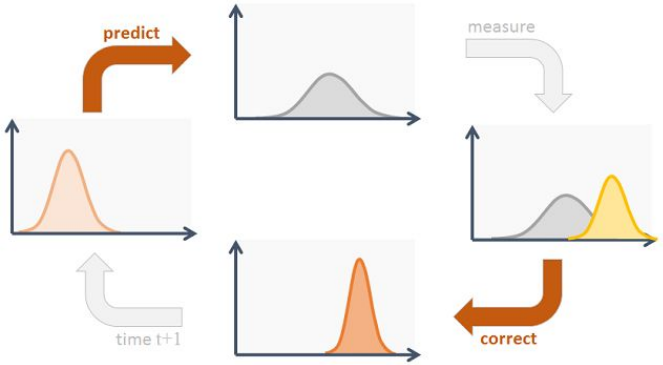
1. SEARCHING WOUNDED
2. GRASPING WOUNDED
3. SEARCHING RESCUE CENTER
4. DROPPING AT RESCUE CENTER

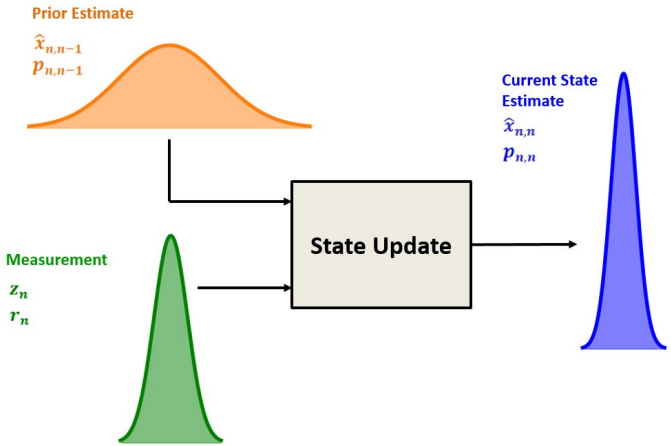
# LOCALIZATION



# LOCALIZATION

- The Kalman Filter (KF) as a **recursive** algorithm to estimate the state of each drone by combining predictions with **noisy** sensor measurements from **Lidar, Semantic** sensors and **odometer** with the **GPS** and **compass**
- This noise has been intentionally introduced in each of the drone sensors. The Kalman Filter **fuses multiple** noisy data sources in a statistically optimal way, continuously refining the system's best estimate of **position, orientation, or velocity** at each time step
- The function is **Pose Estimation Kalman Filter**, used to estimate (**x, y, θ**) for the drone using odometry, GPS, and compass readings. The filter takes the **covariance** in odometry, GPS, compass and LiDAR.



# LOCALIZATION

## Algorithm:

1. **Prediction Step**
   - Compute new state using motion model
   - Compute covariance update
2. **Measurement Update**
   - If GPS is available, update
   - LiDAR update
   - Semantic sensor update

### Prediction Step

The state space equation with **state x**, control **inputs u**, and process **noise w**

$$\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k$$

Current state **Jacobian**

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & -d\sin\theta \\ 0 & 1 & d\cos\theta \\ 0 & 0 & 1 \end{bmatrix}$$

**Updated** state covariance **P**

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q}$$

where **Q** is the **noise covariance**

# LOCALIZATION

## Algorithm:

1. **Prediction Step**
   - Compute new state using motion model.
   - Compute covariance update .
2. **Measurement Update**
   - If GPS is available, update
   - LiDAR update
   - Semantic sensor update

**Measurement Update (GPS & Compass)**

The measurement model H maps the state space to the measurements:

$$\mathbf{y} = \mathbf{z} - \mathbf{H}\mathbf{x}_{k|k-1}$$

Both predicted state and measurement have uncertainties. Hence below are calculated:

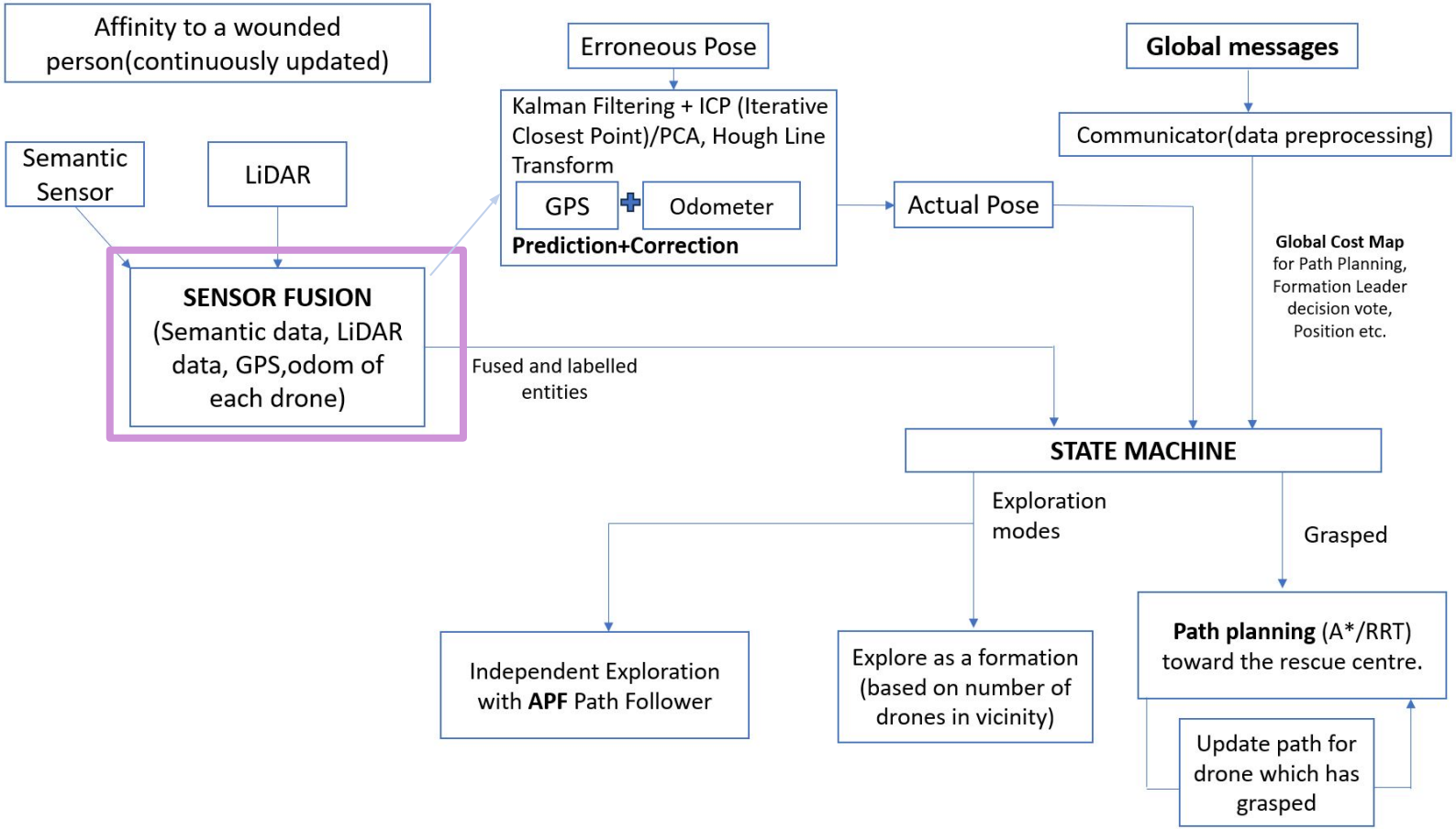
- **y**: Difference between expected and actual measurement termed **innovation**
- **K:** Kalman gain
- Innovation **covariance**

$$\mathbf{K} = \mathbf{P}_{k|k-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \mathbf{R})^{-1}$$

$$\mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}\mathbf{y}$$

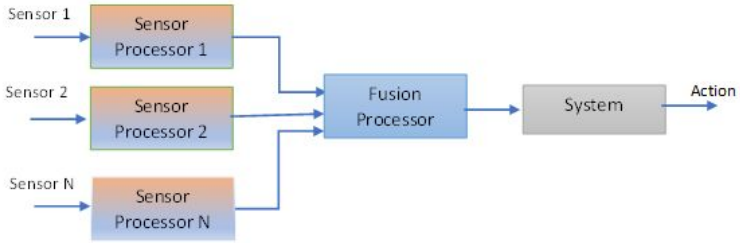
$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}_{k|k-1}$$

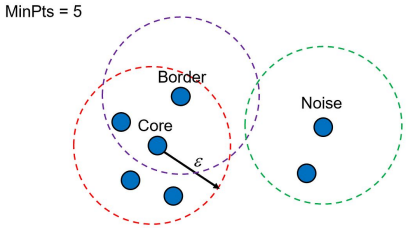
# SENSOR FUSION



# SENSOR FUSION

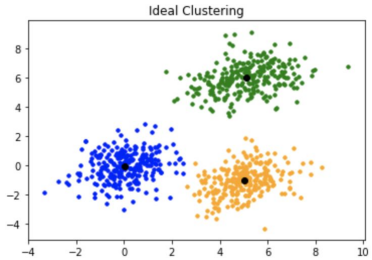
- Sensor fusion combines data from multiple sensors to obtain a more accurate representation of the environment. Combines the sensory data from LiDAR and semantic sensor.
- Filters LiDAR and semantic sensor data to remove noise and converts the stacked sensor data (originally in polar coordinates) to Cartesian coordinates for clustering.
- The clustering was tried out with algorithms like *DBSCAN(Density-Based Spatial Clustering of Applications with Noise)* and K-Means clustering.
- The categories WALL, WOUNDED_PERSON and DRONE are detected, marked and propagated to the entire cluster.
- It outputs the lidar_labels which are output in the Global Occupancy Grid






# SENSOR FUSION

## Label Clustering Methods



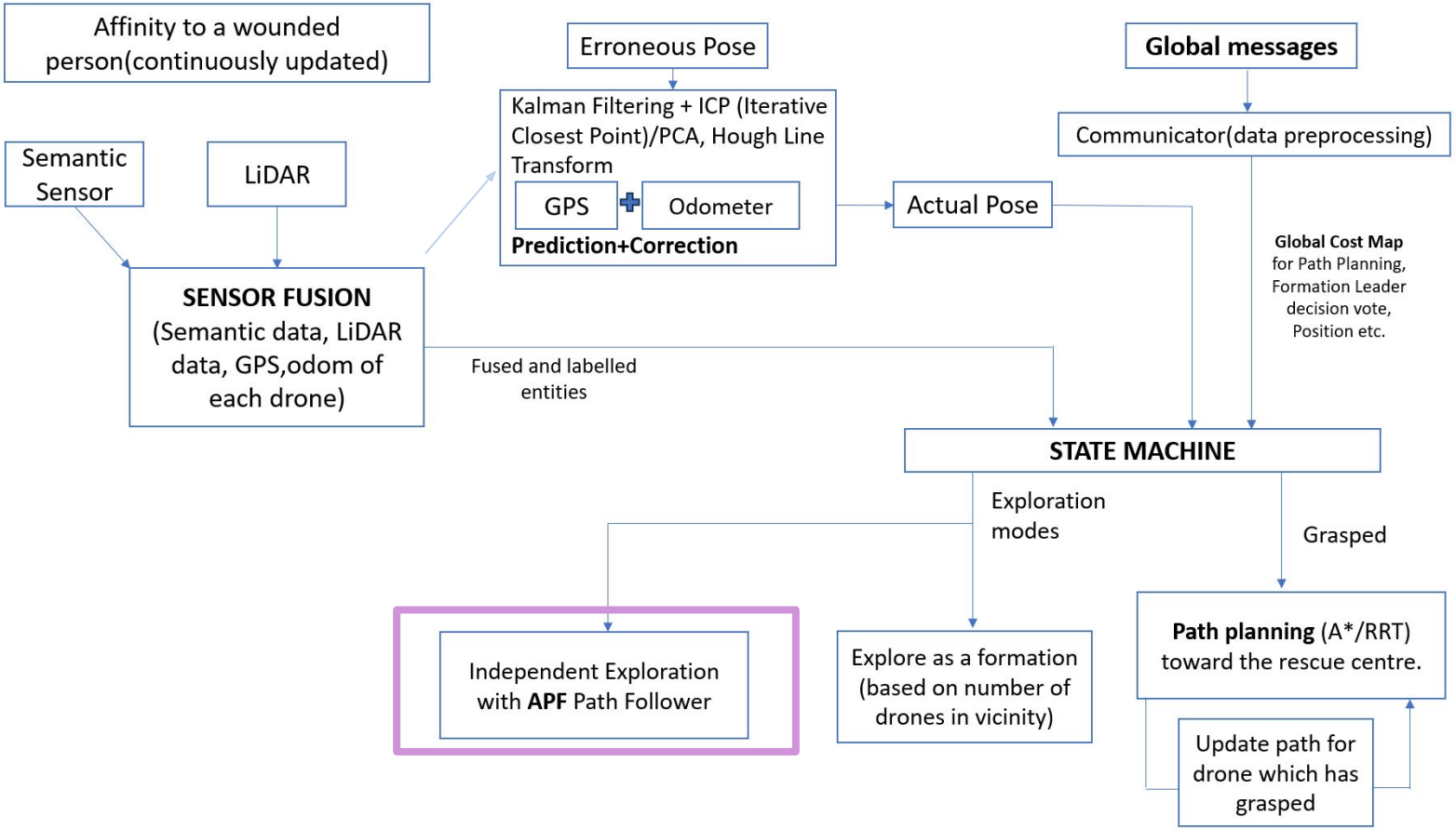

## DBSCAN

- No requirement for specifying number of clusters.
- It can work well with LiDAR and semantic data having noise and outliers, removing them.
- Works better with more realistic drone data which can be clustered based on their density in the feature space.

## K-Means Clustering

- Very sensitive to no.of clusters ‘K’. Needs to be specified.
- Clusters formed in K-Means are spherical or convex in shape. K-Means does not work well with outliers data.
- Assumes that the data is compact and spherical, can’t handle real world irregularities.

# ARTIFICIAL POTENTIAL FIELD

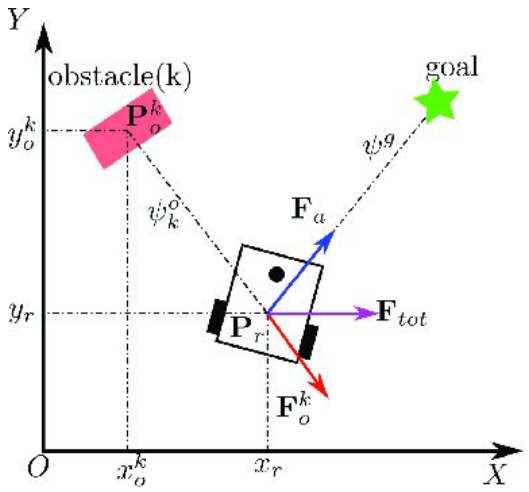


# ARTIFICIAL POTENTIAL FIELD

**Attractive Force ($F_{att}$)**
Guides the drone towards the goal i.e wounded persons,

$$\mathbf{F}_{\text{att}} = K_{\text{att}} \cdot \mathbf{R}^T(\mathbf{g} - \mathbf{p})$$

$K_{\text{att}}$ is the attractive potential gain,
$\mathbf{g} = (g_x, g_y)$ is the goal position,
$\mathbf{p} = (x, y)$ is the current position of the robot.




**Repulsive Force ($F_{rep}$)**
Avoids obstacles(walls, drones, etc) using LiDAR data.

$$F_{\text{rep},i} = -K_{\text{rep},i}\left(\frac{1}{\tilde{d}_i} - 1\right)\frac{1}{\tilde{d}_i^2}$$

$$d_i = \max(\text{lidar_values}_i - d_{\min}, 1 \times 10^{-9})$$

Normalized distances:

$$\tilde{d}_i = \frac{d_i}{d_{\max} - d_{\min}}$$

# ARTIFICIAL POTENTIAL FIELD

A **PID Controller** is used for navigation using APF.

To determine the next goal:

$$\text{dist}_g = \|\mathbf{g} - \mathbf{p}\|$$

If:

$$\textbf{If } dist_g < \epsilon \textbf{ then}$$

move to the next waypoint.

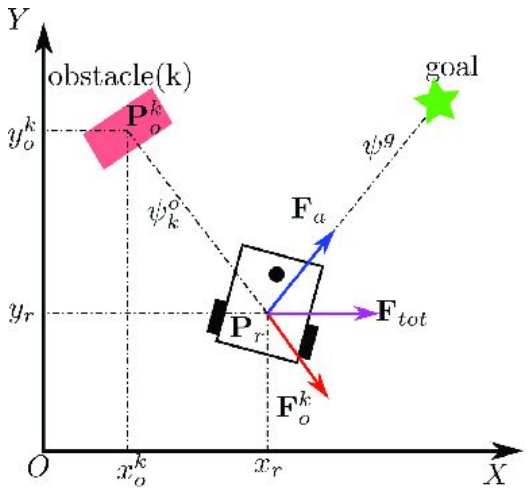
If fewer than 2 waypoints remain, the final goal angle is computed as:

$$\theta_g = \text{atan2}(g_y - g_{y0}, g_x - g_{x0})$$

Orientation error:

$$E_z = \text{normalize}(\theta_g - \theta)$$

Control commands are computed based on these errors.

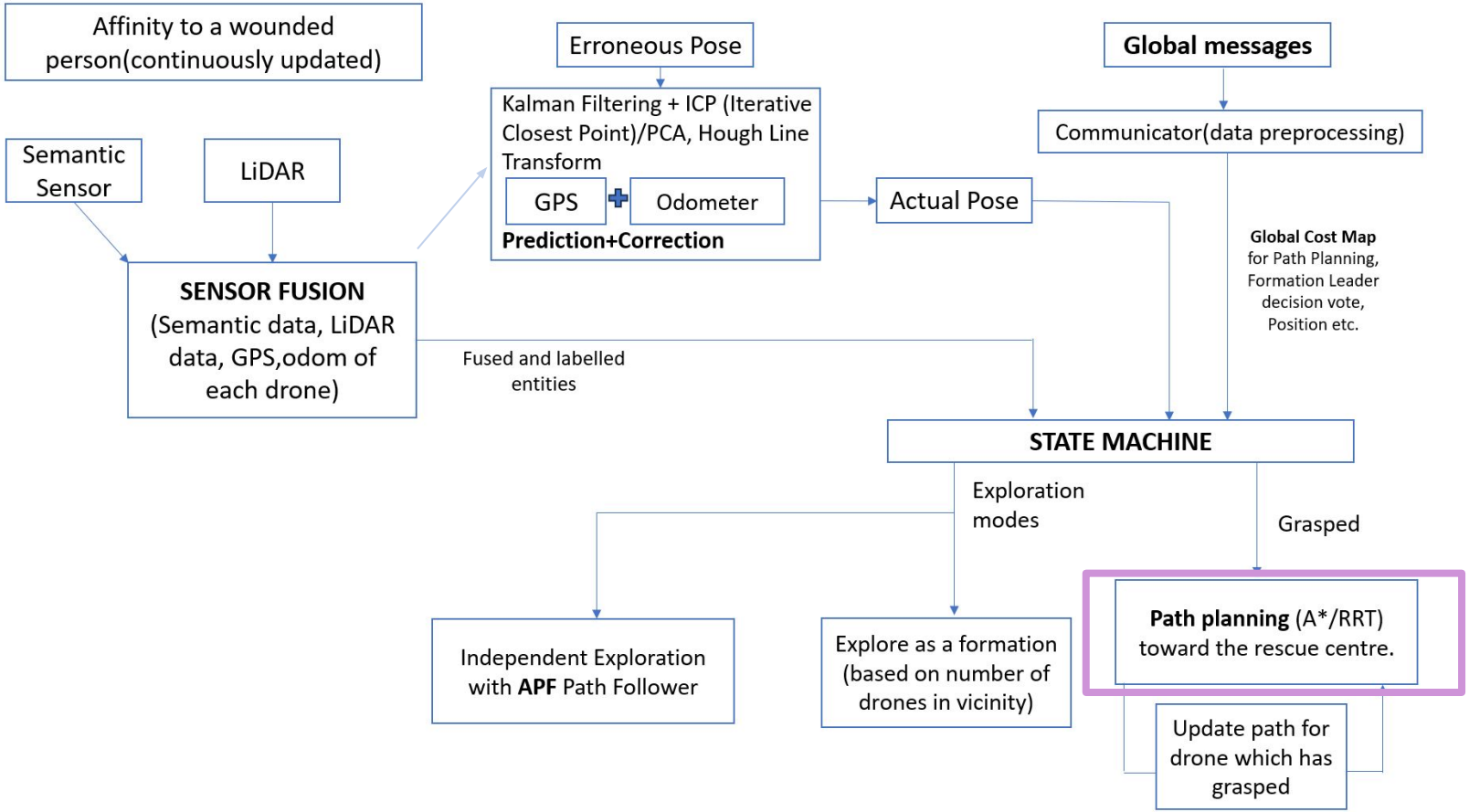


# ARTIFICIAL POTENTIAL FIELD

1. **Path Following**
   - Guides the drone to the target, following a path from the current position to the goal, and keeps the drone on course.
   - Continuously checks the drone state, along with the goal state(s) and computes control commands.
   - If there are only a few waypoints left (two or fewer), the robot prepares to reach the final goal by correcting the orientation alone.
   - Uses lidar based navigation, especially for finding repulsive force, to adjust heading and velocity dynamically.

2. **Command Output**
   - **Returns** `{"forward", "lateral", "rotation", "grasper"}` for movement control based on no.of waypoints to the goal.
   - Switches between **path following** and **goal orientation** based on remaining path length.

# PATH PLANNING



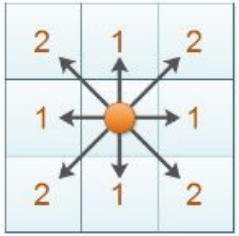
# PATH PLANNING

Uses A* based path planning for each drone, coordinated with voting in case of grasping conflicts

$$f(n) = g(n) + w \cdot h(n)$$

where:

- $f(n)$ is the total estimated cost of the path through node $n$,
- $g(n)$ is the cost from the start node to $n$,
- $h(n)$ is the heuristic estimate from $n$ to the goal,
- $w$ is the weight factor.




Uses **Manhattan Distance** with diagonal adjustments:

$$h(n) = w \times (dx + dy) + (\sqrt{2} - 2) \times \min(dx, dy)$$

Considers **8 possible directions** (including diagonal moves) and checks **grid boundaries** and **obstacles** before adding neighbors.

# PATH PLANNING

**Initialize:**

- Use a priority queue (min-heap) for open nodes.
- Track the cost and parent nodes for path reconstruction.
- Maintain a closed set for visited nodes and a set of remaining goals.

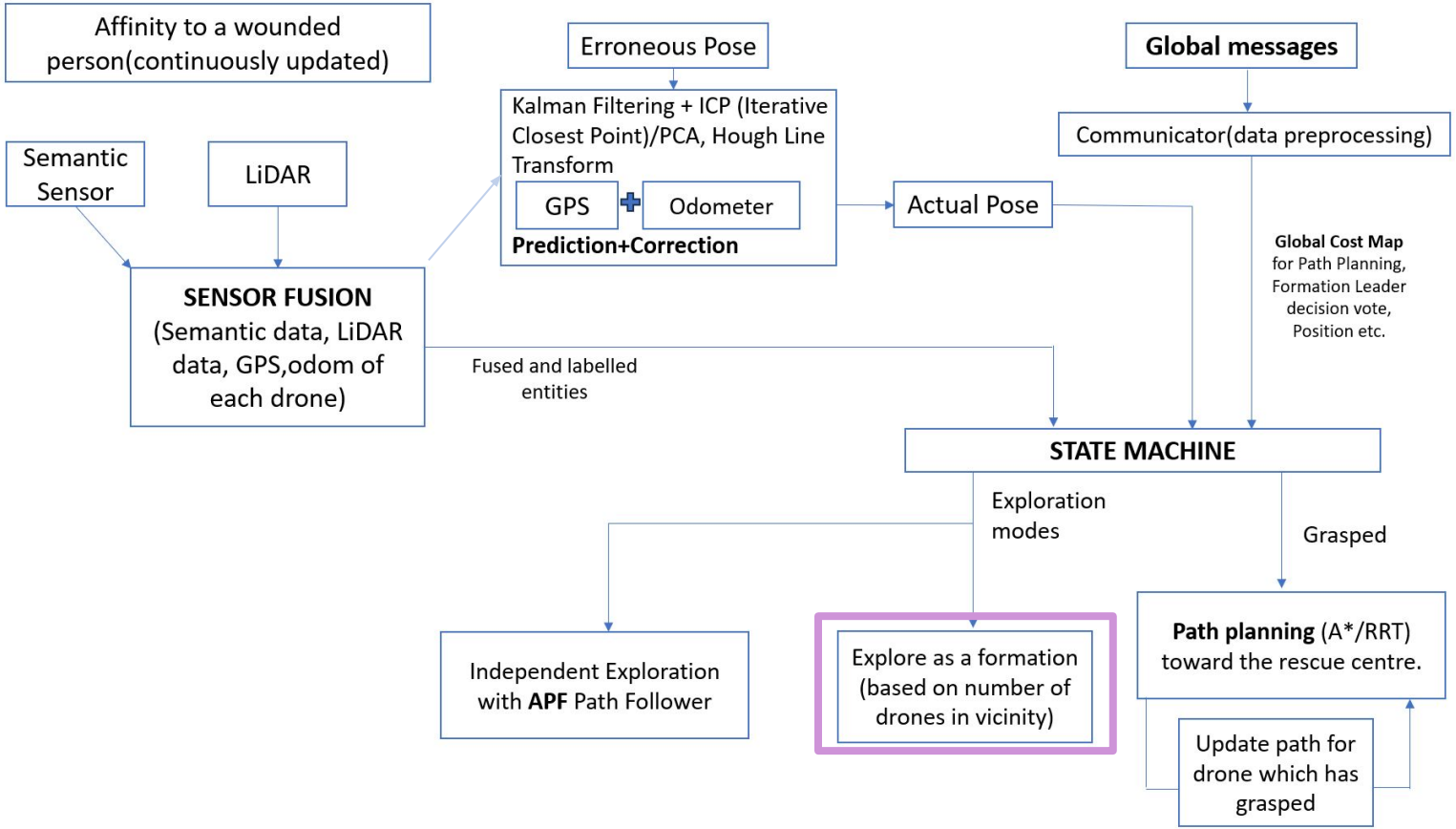
**Pathfinding Process:**

- Extract the node with the lowest cost from the min-heap.
- If the node is a goal, backtrack to reconstruct the path.
- Expand valid neighbors while ensuring diagonal moves are allowed only if adjacent cells are obstacle-free.
- Update costs and heuristics, pushing neighbors into the priority queue.

**Termination:**

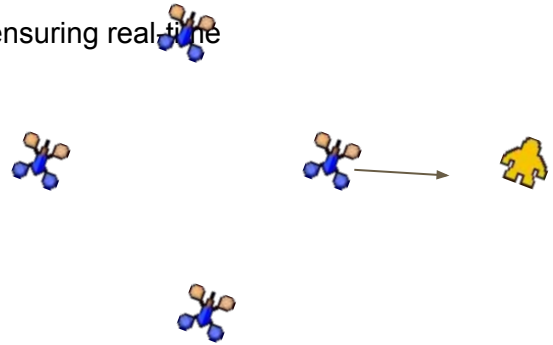
- Stop when all goals are reached or the queue is empty.

# FORMATION CONTROL



# FORMATION CONTROL

- Enables **drone swarms** to move in a **coordinated polygonal formation**, ensuring obstacle avoidance and synchronized movement as a **single unit**.
- Drones are **distributed** in a structured **grid**, with the number of columns computed as: **cols = ceil(sqrt(num_drones))** to maintain compactness and efficiency.
- **Offset adjustments** are applied to center the grid relative to the **leader drone**, ensuring balanced movement.
- **Rotational alignment** is performed to synchronize the formation with the leader's heading for smooth navigation.
- The final **transformed positions** are computed and stored in **positions**, ensuring real-time adaptability.

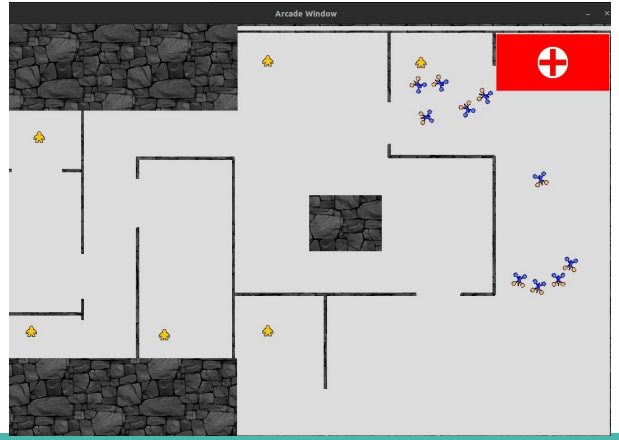


# Numerical Metrics

- **Path Efficiency**: Shortest & smoothest route taken.
- **Obstacle Avoidance Success Rate**: Total no. of collisions per episode
- **Rescue Time**: Time taken to locate and transport back the wounded individuals.
- **Swarm Coordination**: Efficiency of multi-drone collaboration.

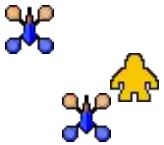
# Numerical Metrics

- Out of these metrics, we were able to test and calculate the 2 main metrics: **Obstacle Avoidance Success Rate** and **Rescue Time**
- **No. of collisions :** An average of 1.5 collisions per 2000 steps tested over 4 maps
- No. of people rescued : An average of 4.5 people per 2000 steps tested over 4 maps



# Challenges Faced

- **Localization Drift**: No GPS Zones led to cumulative errors in odometry-based positioning.
- **Sensor Noise**: LiDAR inaccuracies affected accurate obstacle detection (walls, drones and people).
- **Communication Constraints**: Swarm coordination was limited in no-COM Zones.
- **Finite Life Constraints**: Finite drone life required optimized path planning (eg. to reduce unnecessary collisions)



# REAL LIFE APPLICATIONS

- **Environmental** Monitoring (Wildfire tracking)
- **Agriculture** (Precision Farming, Seed planting and Irrigation)
- Infrastructure Inspection (**Disaster Damage** Assessment)
- Public safety (**crowd control**)
- Entertainment (**Light Shows**) - Formation control
- Integrated functioning of formation of drones to **efficiently manoeuvre** through obstacles and perform **combined tasks** of lifting loads

# FUTURE WORK

- **Optimized Multi-Stage Search & Rescue**
- **Stage 1: Grid Formation** – Drones form a **structured grid** (**cols = ceil(sqrt(num_drones))**) for **systematic scanning**.
- **Stage 2: Adaptive Swarm Search** – On detecting **obstacles or survivor signals**, drones switch to an **AI-driven flexible search** (**Reinforcement learning**:-DQN/PPO , **Swarm Intelligence:-** PSO /ACO)
- **Stage 3: Targeted Rescue Assistance** – **Only one drone** stays with the survivor, relaying **location & status** to rescuers, while others continue searching

# FUTURE WORK

## Global Map Updates for No-Comm Zones

### A. Data Buffering & Store-and-Forward Mechanism

- **Why?** Prevents data loss and ensures synchronization when re-entering a connected zone.
- **How?**
  - Use **local storage (onboard memory)** to save mapping data.
  - Deploy **autonomous relays (drones, mobile ground units)** that periodically collect and transmit data.
  - Implement **ad-hoc mesh networking** (if other units are in range).

# FUTURE WORK

**One Drone as a Stationary Relay at First Waypoint**

- Keep **one drone at the first waypoint** to act as a communication relay.
- The rest of the swarm moves forward, sending updates when in range.
- Helps in **persistent localization** and **efficient information relay**
- This also helps in efficient exploration, identification of kill zones, and helps rescue more wounded entities in the maps.

# REFERENCES


- https://www.cs.cmu.edu/~maxim/classes/robotplanning_sp19/lectures/mgoalastarandida_16350_sp19.pdf
- https://www.youtube.com/watch?v=MN-4WXRwpi0
- https://msl.cs.illinois.edu/~lavalle/papers/Lav98c.pdf
- https://intermediate-and-advanced-software-carpentry.readthedocs.io/en/latest/c++-wrapping.html
- https://ieeexplore.ieee.org/document/8105616

# Thank You

# Future Work

## Minimizing Pose Estimation Errors in No-GPS Zones

### A. Extended Kalman Filter (EKF) / Unscented Kalman Filter (UKF)

- It fuses LiDAR, odometry, and compass data to estimate the state more accurately.
- **Predict Step:** Uses motion model (velocity, acceleration from odometry & compass).
- **Update Step:** Corrects position using LiDAR-based measurements.

**B. Graph SLAM (G2O, GTSAM):** Constructs a pose graph and refines the trajectory.

**ICP (Iterative Closest Point):** Matches LiDAR scans to previous scans more robust to

**LOAM (LiDAR Odometry and Mapping):** High-accuracy LiDAR SLAM, balancing odometry drift with feature extraction.

# Future Work

**Parallelising using OpenMP and MPI, for computational speedup**

- Using OpenMP and MPI communication for parallelising the simulation codes
- Codes involve several loops and conditions which runs faster upon parallelisation
- Wrapping of python codes around C language to achieve parallelisation

**dump everything in the slides first which includes**

1. **overview flowchart**
2. **state machine**
3. **fusion and localisation**
4. **occupancy grid map**
5. **apf (explain a bit differently)**
6. **a star and how graph is built and (don't stress on multiple goals unnecessarily)**
7. **communicated to global map, communication**
8. **rrt for no comm zone (few points in future)**
9. **formation controller (I don't know how to expand)**
10. **comm for swarm and assigning exploration areas by splitting**